ARC-AGI-2
Qwen3.8-27B Experiment Master Plan

Standalone Qwen Architecture -> Controlled Experiments -> NVARC Integration

Target runtime: 4 x NVIDIA L4 | Offline Kaggle | Qwen3.8-27B-FP8 TP=4 | Existing Qwen3-4B NVARC baseline

Master rule: Do not merge every promising solver idea at once. Run one mechanism at a time, keep only mechanisms that add reproducible unique coverage or efficiency, and integrate with NVARC only after the Qwen branch earns promotion.

English master reference | *Priority-ordered* | *Source-audited*

Prepared from the source-verified 12-solver audit, the Qwen3.8 EXP-1 notebook, the EXP0/EDA findings, and the 32.64-LB NVARC notebook/diagnostics.

Executive Decision

Current objective: Build a strong, measurable Qwen3.8-27B generator first. Only then combine it with the existing NVARC Qwen3-4B specialist. The goal is complementary candidate coverage, not replacing NVARC.

- Keep one resident Qwen3.8-27B-FP8 TP=4 engine on the four L4 GPUs. Logical roles are sequential/batched; do not load multiple 27B copies.
- Use the existing EXP-1 direct OFF/XHIGH notebook as the frozen baseline and runtime harness.
- The first causal comparison is PiVOTools-style stateful REPL versus Poetiq-style fresh-call deterministic code repair under matched token/time budgets.
- Do not add a holistic judge until candidate/oracle coverage is clearly higher than selected pass@2. Ranking cannot recover a correct grid that was never generated.
- For NVARC integration, preserve the current NVARC path and leaderboard behavior first. Add Qwen as a separate generator, then test low-risk union strategies before changing attempt_1.
- Route later Qwen compute using label-free task features and NVARC uncertainty/diversity. Never use ground-truth-derived near-miss signals for hidden/private inference.

One-page experiment order

Order	Experiment	Run only when	Decision after run
0	LOAD-Q27	Always first	Runtime stable -> freeze engine
1	DIR-Q27	After load proof	Freeze OFF/XHIGH + parser/serialization baseline
2	PIV-Q27-P0	After direct baseline	Keep only if it adds verified coverage or strong efficiency
3	PTQ-Q27-T0	Paired with PIV	Choose PIV, PTQ, or their union under matched budget
4	LANG-Q27-L0	Only if agentic code adds value	Keep only if it adds different mechanics / unique solves
5	CORE-Q27-C0	Only on object-heavy / collapse cases	Conditional specialist, not global branch
6	LIB-Q27-E0	Only after code branch works	Keep if retrieval cuts iterations or adds coverage without negative transfer
7	JUD-Q27-J0	Only after oracle-selected gap appears	Use judge only if selection, not generation, is the bottleneck
8+	Athanor / Squeeze / Imbue / ARCgentica	Only after earlier ladder is measured	One-by-one late ablations; never merge them simultaneously
Integration	NVARC-B0 -> B4	After Qwen generator promotion	Preserve NVARC attempt_1 first; then increase integration depth

1. Fixed System Constraints and Evaluation Protocol

1.1 Qwen runtime that must remain stable

- Checkpoint: qwen3-8-27b-fp8-repacked-v1, model_type=qwen3_5, FP8 E4M3 dynamic quantization.
- vLLM 0.19.0, tensor_parallel_size=4, distributed_executor_backend="mp", language_model_only=True.
- max_num_batched_tokens=32768, max_num_seqs=16, gpu_memory_utilization=0.90, enforce_eager=True, disable_custom_all_reduce=True, prefix caching disabled in the current baseline.
- Context policy: current notebook derives max_model_len from prompt length and requested generation, bounded between 18,432 and 32,768.
- Reasoning modes already validated by tokenizer/template: enable_thinking=False and enable_thinking=True with reasoning_effort="xhigh".
- Kaggle wall-clock budget: 12 hours with the current notebook reserving 30 minutes. New branches must preserve a similar safety reserve.

1.2 Frozen evaluation progression

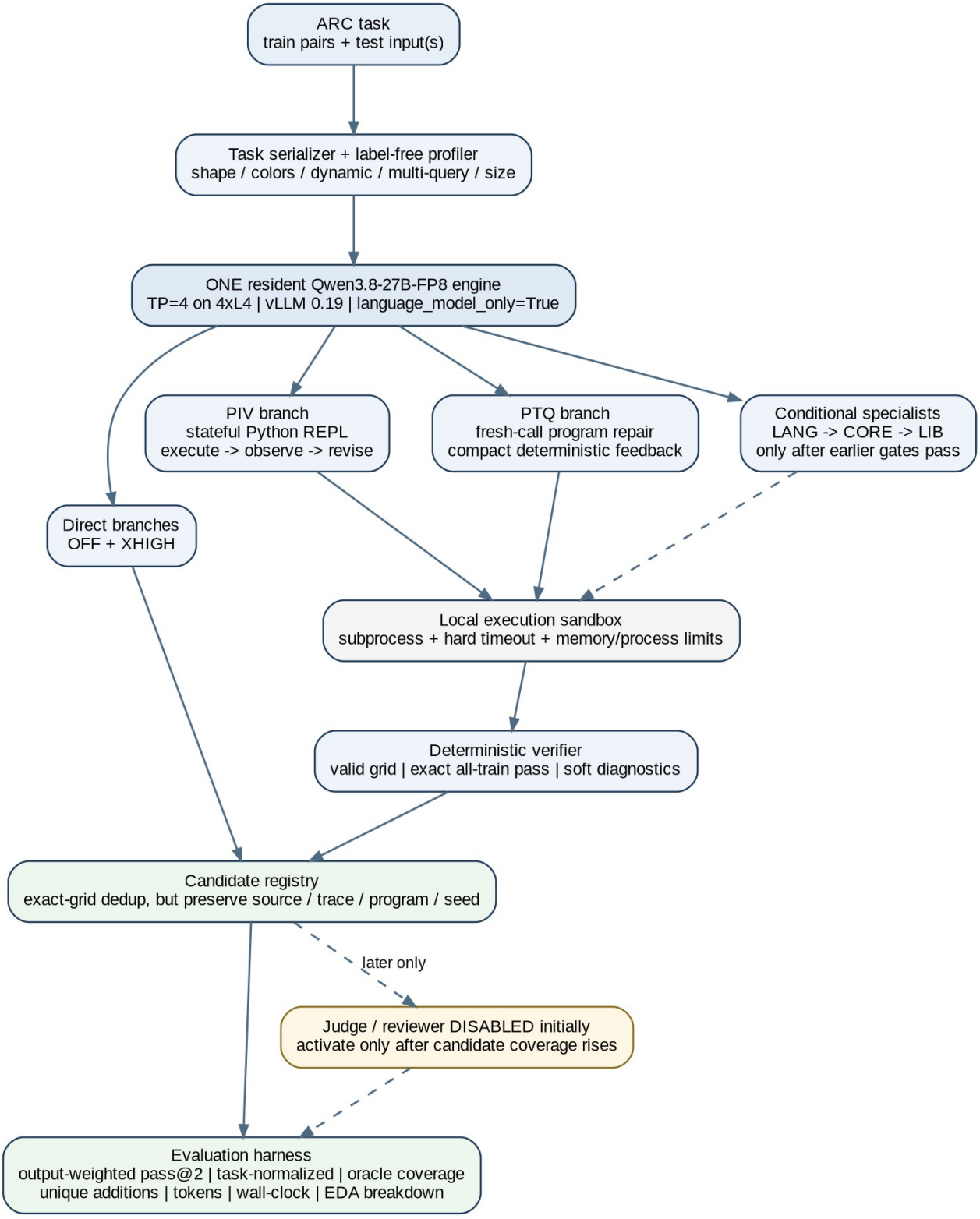
1. Smoke-16: runtime, parsing, tool/sandbox and checkpoint correctness only. Do not use this set to choose a final algorithm.
2. Proxy-48 / frozen hard-proxy set: fast causal ablations with the same task IDs, seeds, time budget and logging schema across variants.
3. Full 120 Public Evaluation: only for promoted variants. Report both current Kaggle output-weighted pass@2 and task-normalized/full-task metrics.
4. EXP0 propensity weighting remains diagnostic only; it is not the official score and must not replace raw metrics.
5. For every Qwen variant, compute NVARC union/oracle statistics offline on the same evaluation outputs before touching the submission integration.

1.3 Mandatory metrics

Metric	Why it matters	Use for promotion?
Output-weighted pass@2	Official 2026 Kaggle denominator	Yes
Task-normalized / full-task score	Historical comparability and multi-query diagnostics	Secondary
GT/oracle in candidate pool	Separates generation failure from selection failure	Yes
Unique additions over NVARC	Primary evidence of complementarity	Yes
Valid-grid rate / parse failures	Measures interface robustness	Yes
All-train-exact program rate	Measures executable hypothesis quality	Yes for code branches
Duplicate ratio / unique candidates	Detects search collapse	Yes
Tokens and wall-clock per test output	Determines 4xL4 feasibility	Yes
EDA segments	Dynamic, multi-query, large/high-ratio, rare-color, hard tiers	Yes
Seed stability	Avoids keeping lucky stochastic branches	Yes on promoted variants

Promotion rule: A mechanism moves forward only if it adds reproducible unique candidate coverage, improves the official/secondary metric, or produces a clear compute-efficiency win. If it only reproduces the same grids at higher cost, drop it.

2. Architecture A - Standalone Qwen3.8-27B Research System



2.1 Architecture principles

- Single-model residency: one TP4 Qwen engine stays loaded. Direct, agent, verifier-review and judge roles are logical stages, not separate GPU-resident models.
- Candidate-first design: every branch writes a structured candidate record immediately. The experiment harness should survive branch failure without losing completed work.
- Execution is authoritative for code branches: final grids come from local program execution, never from prose transcription.

- Exact-grid deduplication does not discard provenance. If NVARC, PIV and PTQ all produce the same grid, keep one grid plus all source/trace/program metadata.
- Judging stays off during early generation experiments. First increase the probability that the correct grid exists in the pool.

2.2 Minimal candidate record

Field	Required	Purpose
task_id / test_index	Yes	Unambiguous output-level scoring
source / branch / seed	Yes	Provenance and independent-family diversity
grid	Yes	Canonical candidate output
valid_grid / shape	Yes	Parser and structural gate
train_exact / train_soft	For programs	Deterministic evidence
hypothesis_summary	Recommended	Compact mechanic trace
program_text / program_hash	For programs	Reproducibility and dedup
tokens / wall_time / tool_calls	Yes	Compute accounting
parent_ids	For refinement/evolution	Lineage tracking
raw_trace_path	Yes for promoted branches	Later holistic judging without bloating live context

3. Priority-Ordered Qwen3.8-27B Experiments

How to use this section: Run the next experiment only after the previous gate is recorded. Do not implement the entire roadmap in one notebook revision.

Q0 / LOAD-Q27 - TP4 load and warmup proof

Priority	Mandatory - first
Purpose	Prove that the exact repacked Qwen3.8-27B-FP8 checkpoint reliably initializes and generates on all four L4 GPUs before any ARC algorithm work.
Minimal implementation	Use the existing SMOKE_GPU path exactly: 4 visible GPUs, vLLM 0.19.0, TP=4, one short OFF generation. Require explicit MODEL LOAD COMPLETE and REAL MODEL WARMUP: PASS markers. Add only minimal TP/NCCL diagnostics if the run stalls.
Run order	SMOKE_GPU only; no full evaluation.
Metrics to log	GPU visibility/memory, load time, warmup time, max_model_len, vLLM/Torch/CUDA versions, first-generation success.
Promotion gate	Three clean runs or one clean Kaggle rerun with stable load/warmup and no engine deadlock.
Stop / rollback rule	Do not proceed to ARC scoring if runtime load is not proven. Fix process/NCCL/vLLM startup first.
Local adaptation	This is our runtime gate, not a solver-paper experiment.

Q1 / DIR-Q27 - Direct OFF vs XHIGH + interface freeze

Priority	Mandatory - second
Purpose	Measure raw Qwen reasoning capability and freeze the parser/serialization interface before adding tools.
Minimal implementation	Run matched direct branches with thinking OFF and XHIGH. Keep candidate outputs separate. Include one controlled formatting ablation only: current strict structured output versus a minimal/tolerant format (CSV or plain grid parser) if parse failure is non-trivial. Do not add agents yet.
Run order	Smoke-16 -> Proxy-48 -> Full-120 only if runtime is stable.
Metrics to log	OFF/XHIGH exact score, union score, valid-grid rate, output length, tokens, latency, EDA segments, NVARC offline union/oracle.
Promotion gate	Freeze the best interface by parse reliability + exact/union coverage, not by prompt aesthetics. Keep both OFF and XHIGH only if they add complementary grids.
Stop / rollback rule	If XHIGH gives no coverage gain and is much slower, keep it only for later conditional routing. If a formatting variant reduces accuracy, revert immediately.
Local adaptation	The prompt/serialization ablation is controlled local testing; CSV was useful in external work but is not assumed to transfer to Qwen.

Primary source anchors: Johan Land paper

Q2 / PIV-Q27-P0 - Stateful Python REPL agent

Priority	Highest-priority algorithm experiment
Purpose	Test whether interleaved execute -> observe -> revise materially improves Qwen3.8 on ARC without changing model weights.
Minimal implementation	One XHIGH lineage. Give Qwen a persistent local Python workspace/REPL with bounded tool calls (start around 6-8). The final answer must be a self-contained solution(grid) or transform(grid) program. Execute in a fresh subprocess and require exact match on every training pair before test execution.
Run order	Proxy-48 first. Full-120 only if proxy coverage and projected wall-clock are acceptable.
Metrics to log	All-train-exact program rate, exact output score, unique NVARC additions, tool calls, repair count, code runtime failures, tokens, wall-clock.
Promotion gate	Promote if it adds reproducible unique NVARC/Qwen-direct coverage or achieves similar coverage with clearly better reliability/compute than direct XHIGH.
Stop / rollback rule	Drop or narrow routing if most programs fail train verification, if it duplicates direct/NVARC outputs, or if projected runtime breaks the 12-hour envelope.
Local adaptation	Use our own local subprocess sandbox and all-train exact gate; do not copy provider-specific tool infrastructure.

Primary source anchors: PiVOTools article | PiVOTools repo

Q3 / PTQ-Q27-T0 - Fresh-call deterministic program repair

Priority	Highest-priority paired experiment
Purpose	Separate the value of persistent agent state from the value of deterministic code feedback.
Minimal implementation	Use fresh Qwen calls for 3-4 refinement rounds. Each round receives the task plus one best partial program and compact deterministic feedback: exact passes, shape errors, cell diff/soft score, runtime error. No persistent chat history. Final program still requires the same exact all-train gate as PIV.
Run order	Exactly the same Proxy-48 subset and matched token/time budget used for PIV.
Metrics to log	Same metrics as PIV plus improvement per refinement round and context size per call.
Promotion gate	Choose PIV, PTQ, or a small union according to unique coverage and compute. Do not choose based on headline score alone.
Stop / rollback rule	If PTQ adds no unique candidates versus PIV, keep only the cheaper/stabler branch. If PTQ wins on reliability, it becomes the default code branch.
Local adaptation	Start far below the released Poetiq 10-iteration budget so PIV-vs-PTQ measures statefulness rather than simply more compute.

Primary source anchors: Poetiq repo

Q4 / CODE-DECISION - Promote one code strategy or a minimal union

Priority	Mandatory decision gate
Purpose	Prevent architecture sprawl. This is the point where one code mechanism earns a place in the main Qwen system.
Minimal implementation	Compare Direct, PIV and PTQ at identical output-level metrics. Inspect unique solve sets, not just total score. Keep one default code branch; keep both PIV and PTQ only if their union adds meaningful non-overlapping coverage.
Run order	No new inference. Re-score saved outputs and traces from Q1-Q3.
Metrics to log	Pairwise overlap, union/oracle gain, unique solve IDs, EDA segment wins, compute per unique added output.
Promotion gate	Exactly one default branch is preferred. A two-branch union is allowed only if each contributes distinct verified outputs.
Stop / rollback rule	If neither PIV nor PTQ adds complementary coverage, stop expanding Qwen agents and move directly to conservative NVARC integration of Direct-Q27 only.
Local adaptation	This gate is our anti-complexity control.

Q5 / LANG-Q27-L0 - Natural-language rule evolution

Priority	Conditional - after code gain
Purpose	Add a code-free hypothesis family that can recover rules when program syntax/search is the bottleneck.
Minimal implementation	Generate a small population of natural-language transformation rules. Use a fresh bounded follower to apply each rule to held-out/train probes. Revise rules from deterministic failure feedback, then apply the best rule to test. Start with 4 initial rules + 2 revisions + 1 synthesis, not hundreds of prompts.
Run order	Proxy-48 hard/near-miss subset first; then full only if unique union gain appears.
Metrics to log	Rule-follow score, unique additions versus code branch and NVARC, follower override rate, tokens, wall-clock.
Promotion gate	Keep only if it contributes mechanics not already covered by Direct/PIV/PTQ.
Stop / rollback rule	Drop if follower reasoning overwhelms instruction quality or if outputs heavily overlap code branches.
Local adaptation	Constrain the follower to rule execution/local ambiguity repair so the experiment measures rule quality more cleanly.

Primary source anchors: Jeremy Berman repo | ARC Prize leaderboard

Q6 / CORE-Q27-C0 - Object/pattern symbolic specialist

Priority	Conditional specialist
Purpose	Target NVARC duplicate-collapse, object-heavy and

	structural-consistency failures with deterministic object extraction and cross-example checks.
Minimal implementation	Extract a compact set of ARC objects/features deterministically (components, colors, bounding boxes, symmetry, relations). Let Qwen propose a bounded number of mechanics, then reject mechanics inconsistent across training examples before generating the test grid.
Run order	Only route to object-heavy / hard / low-diversity-recovery tasks identified without ground truth.
Metrics to log	Unique union gain, object-extraction failure rate, cross-example rejection rate, runtime.
Promotion gate	Keep only as a routed specialist if it adds unique candidate coverage.
Stop / rollback rule	Do not turn the whole system into a large DSL. If the branch rarely adds unique outputs, remove it.

Primary source anchors: CoreThink paper | CoreThink repo

Q7 / LIB-Q27-E0 - Frozen public-program retrieval

Priority	Conditional compute-amortization branch
Purpose	Reduce search cost and provide useful transformation priors without learning from evaluation/private tasks.
Minimal implementation	Build a compact frozen library from public-training programs only. Re-verify every program. At inference, run a small candidate set on the new task training examples, rank by exact-count + cell-soft + shape/runtime validity, dedup behavior/code, and provide top 1-3 diverse programs as priors to the promoted PIV/PTQ branch.
Run order	Offline library build once; then Proxy-48 no-library vs top-1 vs top-3-diverse ablation.
Metrics to log	Iterations/tool calls saved, unique coverage, retrieval relevance, negative transfer, token/time delta.
Promotion gate	Keep if it reduces Qwen search cost or adds unique outputs without hurting previously solved tasks.
Stop / rollback rule	Freeze the library before evaluation/private inference. Do not persist new eval/private solutions across tasks.
Local adaptation	Keep BARC-derived code in a separate licensing bucket and re-verify all imported programs locally.

Primary source anchors: Eric Pang repo | Verified program corpus

Q8 / JUD-Q27-J0 - Holistic trace selector

Priority	Late - only after coverage rises
Purpose	Recover correct minority hypotheses that are already present in the candidate pool.
Minimal implementation	Pool NVARC + Qwen Direct + verified Qwen programs. Exact-grid dedup, retain all provenance/traces. Select at most 6-8 mechanic/provenance-diverse grids. One fresh XHIGH Qwen judge chooses two distinct outputs. Judge synthesis is disabled in J0.
Run order	Run only on tasks where saved-output analysis shows a

	meaningful oracle-versus-selected gap.
Metrics to log	Selector recovery, judge errors, minority-hypothesis wins, pass@2 delta, tokens/time per judged task.
Promotion gate	Keep only if it closes a measurable fraction of the oracle gap without damaging high-confidence cases.
Stop / rollback rule	Do not use judge compute to hide weak generation. If oracle coverage is low, return to generation experiments.

Primary source anchors: Johan Land paper | Johan Land repo

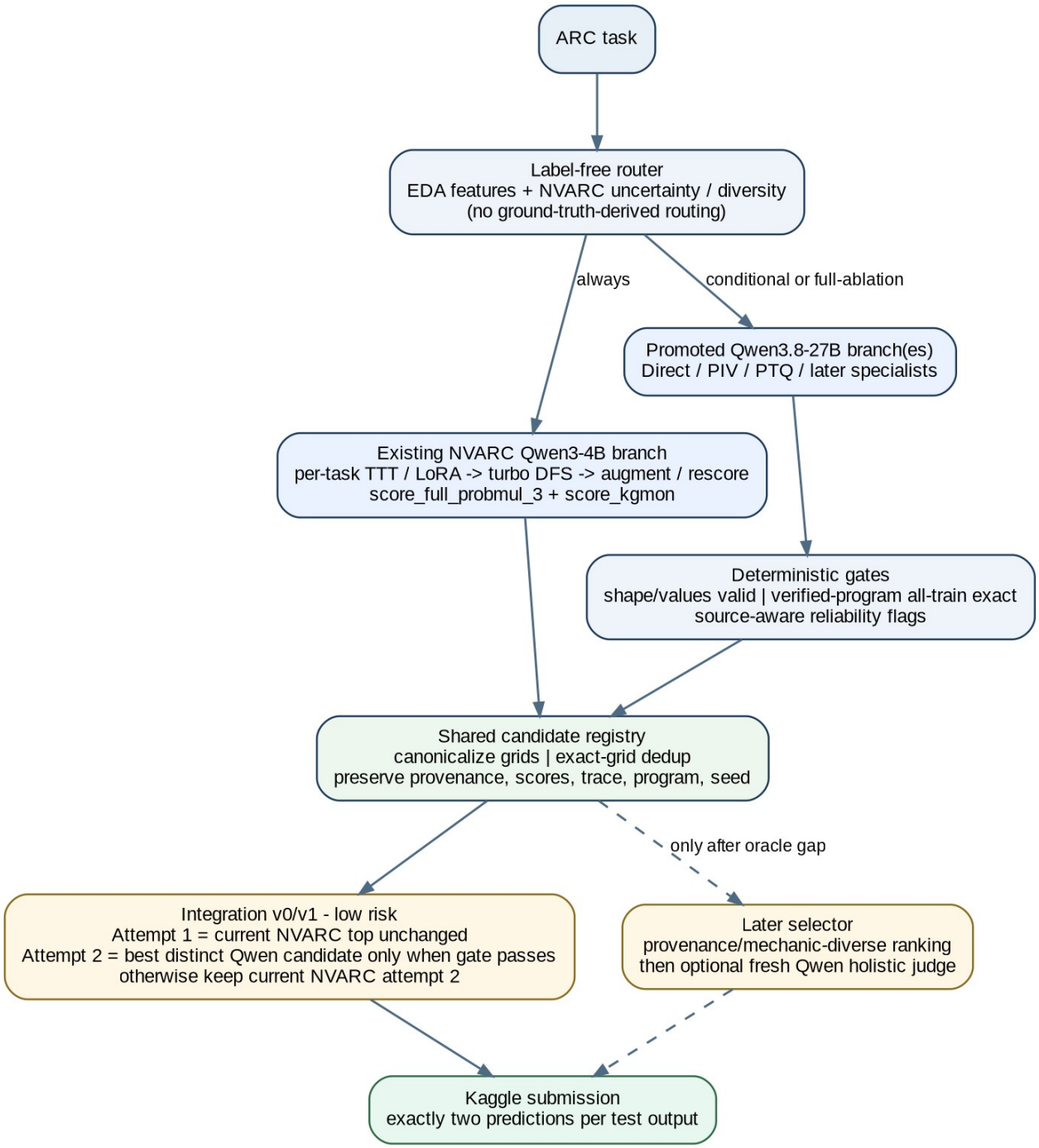
3.10 Deferred late-stage ablations - run one at a time, only if triggered

Mechanism	Trigger	Minimal experiment	Why it is not earlier
Athanor reviewer	Many train-perfect programs fail by ambiguous generalization	Fresh artifact-only Qwen review: APPROVE / REJECT / EXPAND_CANDIDATES	Adds extra inference; useful only after train-perfect candidates exist
Squeeze-Evolve routing	Compute is the bottleneck and candidate disagreement predicts difficulty	Consensus=free carry; disagreement -> stronger Qwen budget / recombination	Routing optimization should follow, not precede, a useful generator
Imbue evolution	Single-lineage repair stalls in local optima	Small population + failure-aware mutation + novelty-weighted parent choice	Population search is expensive on 4xL4
ARCgentica delegation	Hard tasks show clear decomposable subproblems	Parent Qwen calls 1-2 scoped specialist analyses only when uncertain	Fixed multi-agent fan-out is too expensive; adaptive delegation is a late optimization

Source anchors: Athanor | Squeeze-Evolve | Imbue Darwinian Evolver | ARCgentica

4. Architecture B - Integration into the Existing Qwen3-4B NVARC Notebook

Integration principle: Treat NVARC as the current specialist generator and Qwen3.8 as a separate complementary generator. Do not rewrite the NVARC core until the union analysis proves Qwen adds value.



4.1 Why this architecture matches our notebook findings

- The 31-experiment diagnostic notebook found GT present in only 55/172 output candidate pools (32.0%) in that analysis, so candidate coverage is a primary bottleneck.
- The same diagnostic showed a current selected diagnostic score around 33.50 versus an oracle of 41.17 from the existing pool: there is selection headroom, but not enough to justify solving selection before generation.

- Average raw-candidate duplication was about 37.8%, with some tasks much higher, indicating search budget collapse.
- The two NVARC scorers were highly correlated (about 0.961 rank correlation in that experiment), so a genuinely different Qwen generator is more valuable than another near-identical ranker.
- Therefore Qwen should first be evaluated by unique NVARC-union additions, not by whether it beats NVARC standalone.

4.2 Preserve the current NVARC branch

- Keep the existing Qwen3-4B ARC-specialized path: task-time adaptation/LoRA, turbo DFS generation, augmentation scoring and current candidate decoding.
- Preserve score_full_probmul_3 and score_kgmon outputs as source features even if final selection later changes.
- Do not alter the current 32.64-LB reported baseline notebook during B0 union analysis. Generate Qwen outputs separately first.
- The existing run_dual_selection behavior should not be the final hybrid selector; the hybrid system must explicitly distinguish NVARC rank-2 from a genuinely different Qwen hypothesis.

4.3 Integration stages - one by one

Stage	Notebook change	Attempt 1	Attempt 2	Promotion condition
B0 - Offline union only	No submission change. Import saved Qwen candidates into an analysis cell.	Current NVARC	Current NVARC	Qwen adds unique oracle/GT coverage on frozen eval outputs.
B1 - Safe hybrid slot 2	Add Qwen candidate registry + gate.	Current NVARC top unchanged	Best distinct gated Qwen candidate; fallback=current NVARC attempt 2	No regression on stable NVARC cases; measurable unique recovery.
B2 - Conditional Qwen router	Run Qwen only on label-free hard/uncertain tasks.	Current NVARC top	Hybrid distinct candidate	Projected 12h runtime becomes feasible without losing B1 gains.
B3 - Source-aware selector	Rank union candidates by provenance + reliability + NVARC scores.	Best source-aware candidate	Best mechanic-diverse nonduplicate	Improves selected score while preserving oracle coverage.
B4 - Holistic judge	Add JUD-Q27-J0 only on ambiguous tasks.	Judge choice 1	Judge choice 2	Closes a material oracle-selected gap with acceptable compute.

4.4 Label-free routing features for hidden/private inference

- Task structure: dynamic output shape, multi-query count, input/output size ratio inferred from training pairs, grid size, color count/rarity, separators/subgrids and object count.
- NVARC search signals: number of unique decoded grids, duplicate ratio, score margin between top candidates, disagreement between score_full_probmul_3 and score_kgmon, beam/search exhaustion, invalid-grid rate.
- Qwen program signals: whether a verified all-train program was found, number of independent train-perfect mechanics, program/runtime stability.
- Do not route using public-eval ground-truth similarity or "near miss to GT" because those labels do not exist in the competition test rerun.

4.5 Low-risk first hybrid selector

Recommended B1 rule: Keep NVARC attempt_1 exactly unchanged. Replace attempt_2 only when Qwen supplies a valid, distinct candidate that passes the branch-specific gate (for programs: exact all-train verification). Otherwise retain the current NVARC attempt_2. This isolates complementarity with minimal leaderboard risk.

4.6 Candidate pool merge contract

6. Canonicalize every grid to an immutable tuple-of-tuples representation and validate values 0-9 / rectangular shape.
7. Deduplicate exact grids, but merge provenance arrays rather than discarding duplicate sources.
8. Keep NVARC scores, Qwen source, seed, program verification, trace summary and runtime in the same record.
9. For multi-query tasks, select two outputs independently per test output while retaining task-level provenance for analysis.
10. Write merged-candidate JSON immediately so later selector experiments can be rerun without repeating Qwen inference.

5. Practical One-by-One Workflow

Order	Action	Decision
Step 1	Run Q0 load/warmup.	If unstable: stop and fix runtime. If stable: freeze engine config.
Step 2	Run Q1 Direct OFF/XHIGH and freeze parser/interface.	Save every candidate and compute NVARC offline union; do not integrate yet.
Step 3	Run Q2 PIV on Proxy-48.	If no verified unique gain: do not scale it globally.
Step 4	Run Q3 PTQ on the same Proxy-48 with matched budget.	Choose PIV, PTQ or small union at Q4.
Step 5	Run B0 offline NVARC union using saved outputs.	If Qwen adds no unique coverage, stop architecture expansion and revisit prompts/model capability.
Step 6	If code branch adds value, run Q5 LANG.	Keep only if it adds a new failure-recovery family.
Step 7	Run Q6 CORE or Q7 LIB only when their trigger segment exists.	These are routed specialists, never default global branches initially.
Step 8	Run B1 low-risk hybrid attempt_2.	Attempt_1 remains the 32.64-LB NVARC behavior.
Step 9	If runtime is too high, add B2 conditional routing.	Use label-free EDA/NVARC uncertainty signals.
Step 10	Only if oracle-selected gap is now meaningful, run Q8/B4 judge.	Selection comes after generation coverage.
Step 11	Only after the core pipeline is stable, test one late mechanism at a time.	Athanor review -> Squeeze routing -> Imbue evolution -> ARCgentica delegation, only if a specific bottleneck justifies it.

Do not do this: Do not build a single notebook containing PIV + PTQ + LANG + CORE + LIB + evolution + multi-agent delegation + judge before any one of them is measured. That makes failures impossible to attribute and will likely exceed the 4xL4 wall-clock budget.

5.1 Notebook/versioning discipline

- Create one notebook version per promoted experiment family, with the previous version kept runnable.
- Every run writes a manifest containing git/notebook hash, model path, vLLM config, task IDs, seeds, token caps and wall-clock budget.
- Separate generation artifacts from selector artifacts. A new selector should be able to re-score saved candidate pools without re-running the model.
- Never silently change the evaluation denominator. Store output-weighted pass@2, task-normalized and oracle coverage side by side.

5.2 Suggested file/module boundary

```
q27_runtime.py          # vLLM/TP4 load, sampling profiles
q27_format.py           # serialization + tolerant grid parser
q27_direct.py           # OFF/XHIGH generator
q27_tools.py            # subprocess sandbox + deterministic verifier
q27_piv.py              # stateful REPL agent
q27_ptq.py              # fresh-call refinement
q27_specialists.py      # LANG / CORE / LIB (conditional)
candidate_store.py      # canonical grids + provenance + checkpoints
metrics.py              # Kaggle + task-normalized + oracle/union metrics
nvarc_bridge.py         # import NVARC candidate pools and scores
hybrid_selector.py      # B1/B3 selector; judge disabled by default
q27_judge.py            # late JUD-Q27-J0 only
```

6. Primary Source Map

- Qwen3.8-27B-FP8 official model
- NVARC
- Confluence Labs ARC-AGI-2
- Squeeze-Evolve
- Athanor
- Imbue Darwinian Evolver
- Noemon
- Symbolica ARCgentica
- PiVOTools Agentic Coding article
- PiVOTools ARC repo
- Poetiq ARC solver
- Johan Land - Modality-Driven Search
- CoreThink
- Jeremy Berman - evolutionary NL instructions
- Eric Pang - evolutionary program synthesis
- Verified ARC-AGI-2 public-training programs
- ARC Prize Community Leaderboard
- Kaggle ARC Prize 2026 competition

7. Promotion Checklist for Every Experiment

- ☐ Same frozen task subset and same evaluation denominator as the comparison run.
- ☐ Candidate files, provenance and traces saved before selection.
- ☐ Official output-weighted pass@2 + task-normalized + oracle/union metrics recorded.
- ☐ Unique NVARC additions listed by task/test-output ID.
- ☐ EDA breakdown recorded, especially multi-query, dynamic, large/high-ratio and hard tiers.
- ☐ Tokens, tool calls and wall-clock projected to the full Kaggle rerun.
- ☐ At least one rerun/seed check before promoting a stochastic method.
- ☐ If no unique coverage or efficiency benefit: remove the branch rather than keeping it "just in case".
- ☐ Only after a branch is promoted may it enter the NVARC hybrid notebook.

Final target: A small, auditable hybrid: NVARC remains the ARC-specialized 4B generator; Qwen3.8-27B contributes only the branches that prove complementary coverage under 4xL4; the selector uses two attempts efficiently; every additional mechanism has a measured reason to exist.